

Guía del Profesor — Aspectos Avanzados de Programación Funcional

Referencia docente — Tema 04

Matías Gel

Ciclo lectivo 2026

Índice

1	Guía del Profesor — Tema 04	1
1.1	Aspectos Avanzados de Programación Funcional	1
1.2	Resumen Ejecutivo	1
1.3	Índice de Artefactos	2
1.4	Plan de Clase Detallado	3
1.5	Extractos clave para enfatizar en clase	4
1.6	Guion de intervención docente (con frases literales)	6
1.7	Preguntas de evaluación formativa (con respuestas esperadas)	7
1.8	Riesgos frecuentes y mitigación	8
1.9	Material de respaldo	9

1 Guía del Profesor — Tema 04

1.1 Aspectos Avanzados de Programación Funcional

Elaborada por: Dr. Roberto (class-writer) **Materia:** Paradigmas y Lenguajes de Programación 2026 — UNTDF / IDEI — IF020 **Año:** 4° Licenciatura en Sistemas
Semana: 3 — Clase 1 **Duración:** 120 minutos

1.2 Resumen Ejecutivo

Esta clase profundiza programación funcional con enfoque aplicado y comparativo entre TypeScript y Clojure. El eje central es mostrar cómo las mismas ideas (composición, modelado explícito de errores y separación de efectos) se expresan con herramientas distintas pero principios comunes.

Mensaje central de clase:

El objetivo no es memorizar sintaxis, sino aprender a diseñar programas con menos estado mutable, flujos más explícitos y mejor trazabilidad de errores.

Estructura en un vistazo:

Bloque	Tiempo	Qué logra
A — Fundamentos avanzados	35 min	Consolida composición e inmutabilidad en TS/Clojure
B — Abstracciones y efectos	35 min	Introduce Result/Option y transducers con criterios de uso
C — Concurrencia y metaprogramación	30 min	Compara canales, STM, agents y promesas
D — Taller y cierre	20 min	Integra conceptos en resolución comparativa guiada

Artefactos de clase:

- filminas.md — F-01 a F-42
- minuta.md — guion completo por filmina
- guia-estudio.md — documento de estudio autónomo para alumnos

1.3 Índice de Artefactos

Archivo	Descripción	Uso en clase
diseno.md	Alcance pedagógico y restricciones de contenido	Control de scope (evitar desvíos)
minuta.md	Guion por filmina con tiempos y transiciones	Documento operativo principal durante dictado
filminas.md	Plan de presentación con tipo de filmina y código	Soporte visual de clase
guia-estudio.md	Desarrollo expandido para alumnos	Preclase, repaso y recuperación
guiaprofesor.md	Síntesis ejecutiva docente	Repaso rápido previo al dictado

1.4 Plan de Clase Detallado

1.4.1 BLOQUE A — Fundamentos avanzados

Tiempo: 35 minutos | **Filminas:** F-01 a F-13

Objetivo del bloque: Conectar fundamentos funcionales con prácticas concretas de diseño y lectura de pipelines.

- F-01 a F-03: Apertura y contraste imperativo/funcional.
- F-04 a F-06: Funciones puras, inmutabilidad y pipeline en TypeScript.
- F-07 a F-10: Semántica de filter/map/reduce + composición reusable.
- F-11 a F-13: Puente conceptual a Clojure (laziness + persistencia).

Preguntas guía del bloque:

- Qué ganamos al pasar de pasos mutables a transformaciones declarativas.
 - Qué parte del pipeline explica mejor la intención del programa.
-

1.4.2 BLOQUE B — Abstracciones y efectos

Tiempo: 35 minutos | **Filminas:** F-14 a F-24

Objetivo del bloque: Enseñar modelado explícito de estados y errores, y cuándo optimizar composición.

- F-14 a F-16: ADT y Result en TS como contrato de flujo.
- F-17 a F-21: Option/Maybe y transducers en Clojure.
- F-22 a F-24: API funcional genérica + funciones de orden superior + macros (visión de diseño, no profundidad de implementación).

Punto de cuidado didáctico:

No mezclar discusión sintáctica con discusión conceptual. Primero el problema, después la herramienta.

1.4.3 BLOQUE C — Concurrencia y metaprogramación

Tiempo: 30 minutos | **Filminas:** F-25 a F-34

Objetivo del bloque: Comparar decisiones de concurrencia según tipo de flujo y nivel de coordinación.

- F-25 a F-29: Modelo Clojure (core.async, STM, agents).

- F-30 a F-33: Modelo TypeScript (Promise y async-await).
- F-34: Síntesis en diseño de flujo continuo.

Preguntas guía del bloque:

- Cuándo un canal representa mejor el problema que una promesa.
 - Qué parte debe permanecer pura aunque el sistema tenga I/O.
-

1.4.4 BLOQUE D — Taller comparativo y cierre

Tiempo: 20 minutos | **Filminas:** F-35 a F-42

Objetivo del bloque: Integrar en una misma actividad modelado de datos, composición y manejo explícito de error.

- F-35: Presentación del desafío.
 - F-36/F-37: Guiones base TS y Clojure.
 - F-38: Puesta en común comparativa.
 - F-39/F-40: Preguntas de cierre + evaluación rápida.
 - F-41/F-42: Resumen y puente al tema siguiente.
-

1.5 Extractos clave para enfatizar en clase

1.5.1 1) Transformación funcional en TS

Frase ancla para la clase: > “Un pipeline no es una forma de ahorrar líneas. Es una forma de decir exactamente lo que el programa hace, sin ocultar pasos.”

Qué resaltar: - Mostrar el mismo código imperativo y declarativo lado a lado. Preguntar cuál es más fácil de leer en 6 meses. - `.filter()` comunica *intención* (selección por criterio). Un `for` con `if` no lo hace explícito. - Cada método del pipeline puede testearse con un array de dos elementos. No necesita contexto de la aplicación completa. - Señalar que si necesitamos debuggear, podemos insertar un `.map(x => { console.log(x); return x; })` sin romper nada.

Código de contraste para la pizarra:

```
// Imperativo: flujo oculto en variables mutables
let suma = 0;
for (const o of ordenes) {
  if (o.activa && o.total > 100) suma += o.total;
}
```

```
// Pipeline: intención explícita
const suma = ordenes
  .filter(o => o.activa)
  .filter(o => o.total > 100)
  .map(o => o.total)
  .reduce((acc, t) => acc + t, 0);
```

1.5.2 2) Result y Option

Frase ancla para la clase: > “Una excepción no capturada es un contrato roto. Un **Result** es un contrato explícito escrito en el tipo.”

Qué resaltar: - La firma `dividir(a, b): number` miente — puede lanzar. La firma `dividir(a, b): Result<number, string>` es honesta. - Con **Result**, el compilador *obliga* al consumidor a manejar el caso de error antes de acceder al valor. No hay acceso accidental. - **Option/Maybe** es **Result** sin razón de error. Útil para búsquedas: `buscarUsuario(id): Maybe<Usuario>` en lugar de `Usuario | null`. - Error frecuente en alumnos: mezclar **Result** con **throw**. Explicar que dentro de una función que retorna **Result** **nunca** se lanza excepción — se retorna `err()`.

Pregunta de comprobación rápida: “Si `validar(x)` retorna `Result<Order, string>`, ¿cómo accedo a `Order.total`?” (Respuesta esperada: verificar `.ok === true` antes.)

1.5.3 3) Transducers

Frase ancla para la clase: > “Un transducer no es una optimización. Es una abstracción: describe *qué hacer* sin decir *sobre qué*.”

Qué resaltar: - Mostrar que el mismo `xf` funciona con **transduce**, con **into**, con un canal de `core.async`. La transformación no depende del contenedor. - El beneficio de rendimiento (sin colecciones intermedias) es consecuencia, no propósito primario. - **Cuándo NO usar transducers:** arrays pequeños, pipelines de 2 pasos, código que otros deben mantener sin conocer Clojure avanzado. Aplicar cuando el pipeline tiene 3+ pasos y el volumen de datos es real. - En TypeScript: no hay transducers nativos, pero la idea de **compose** de transformaciones aplica igual. Librerías como **ramda** o **transducers-js** los implementan.

1.5.4 4) Concurrency

Frase ancla para la clase: > “El problema de concurrency no es velocidad. Es correctitud. ¿Qué garantías tenemos de que dos procesos no ven versiones inconsistentes del mismo dato?”

Cuadro de decisión para presentar:

Necesidad	Modelo recomendado
Un resultado que llega una vez (API call)	Promise / <code>async-await</code>
Eventos continuos (teclado, websocket, sensores)	Canal / <code>core.async</code> / <code>AsyncGenerator</code>
Actualizar múltiples valores coordinados atómicamente	<code>ref</code> + <code>dosync</code> (STM)
Actualización asíncrona desacoplada	<code>agent</code>
Estado simple con actualizaciones frecuentes	<code>atom</code>

Qué resaltar: - La inmutabilidad no *soluciona* la concurrency, la *simplifica*: elimina la categoría de error de estado compartido mutable. - Separar siempre la lógica pura (sin I/O) de los efectos. La lógica pura puede ejecutarse en cualquier thread sin coordinación. - En JavaScript/TypeScript: no hay threads reales, pero el event loop más `async/await` introduce puntos de reentrada. Un `await` es un punto donde otro callback puede ejecutarse.

1.6 Guion de intervención docente (con frases literales)

1.6.1 Apertura de clase (F-01)

“Hoy no aprendemos Clojure. Aprendemos a diseñar programas con menos estado mutable y más composición. Clojure y TypeScript son los vehículos para ver cómo la misma idea se expresa distinto.”

1.6.2 Antes de cada ejemplo de código

“Antes de correrlo: ¿qué esperan que devuelva? Díganme el tipo, no el valor.”

- Si el grupo duda, simplificar la entrada (1 elemento en vez de 5).
- Si aciertan, complicar: ¿qué pasa si la colección está vacía?

1.6.3 Al introducir comparativas TS vs Clojure

“Busco que identifiquen qué idea es la misma en ambos. La sintaxis cambia; el problema que resuelven no. ¿Qué concepto vemos acá que ya vimos en TypeScript?”

1.6.4 Al mostrar Result (F-14 a F-16)

“¿Quién leyó código que tiene `try-catch` anidados de 3 niveles? Eso es exactamente lo que evitamos con este patrón.” - Preguntar: “Si la función devuelve `Result<number, string>`, ¿puedo escribir `resultado + 1`?” (No — debo verificar `.ok` primero.)

1.6.5 Durante el taller (F-35)

“No me importa si el código corre. Me importa que el tipo de `clasificarOrden` sea correcto y que el pipeline sea legible.” - Si un grupo termina temprano: pedirles que agreguen una tercera categoría de error y refactoren sin romper el pipeline. - Si un grupo está trabado: preguntar “¿cuál es el tipo de la función? Trabajen desde el tipo hacia la implementación.”

1.6.6 Cierre (F-41)

“La pregunta que quiero que se lleven no es ‘cómo funciona `transduce`’. Es: ‘dado un problema, ¿qué modelo me da más control sobre los errores y el flujo?’”

1.6.7 Señal de ritmo

- Si el Bloque A lleva más de 37 minutos: comprimir F-10 o F-11, no omitirlos.
- Si el Bloque B lleva más de 37 minutos: omitir la comparación detallada de transducers vs pipeline (F-21) y referirla a la guía de estudio.
- El taller no tiene que terminar: lo importante es que discutan y lleguen a la puesta en común (F-38).

1.7 Preguntas de evaluación formativa (con respuestas esperadas)

P1: ¿Qué diferencia conceptual hay entre `reduce` y un `for` que muta un acumulador?

> Respuesta esperada: En `reduce`, el acumulador es un **parámetro nuevo** en cada invocación — no hay variable que se modifique. El `for` muta la variable `acc` en cada paso. La diferencia no es de resultado sino de garantías: `reduce` con función pura garantiza que no hay estado externo afectado.

P2: ¿Qué ventaja tiene modelar error como datos (`Result`) frente a `throw`?

> Respuesta esperada: Con `throw`, el error es invisible en la firma de la función — quien llama puede no saber que puede fallar. Con `Result<T, E>`, el tipo comunica explícitamente los dos caminos. El compilador de TypeScript puede verificar que se manejan ambos. Facilita composición (no hay `try-catch` para encadenar).

P3: ¿En qué escenario un transducer aporta valor real frente a un pipeline con `.filter().map()`?

> Respuesta esperada (al menos dos de): dataset grande donde las colecciones

intermedias tienen costo de memoria; pipeline que se aplica sobre múltiples fuentes (array, stream, canal) sin reescribir; cadena de 4+ transformaciones donde el rendimiento es medido y relevante. **No aplica** en arrays de decenas de elementos o pipelines de 2 pasos.

P4: ¿Cuándo elegirías un canal (`core.async`) en lugar de una `Promise`? > Respuesta esperada: Cuando la fuente emite **múltiples valores** a lo largo del tiempo (eventos de teclado, mensajes de websocket, lecturas de sensor). `Promise` resuelve una única vez. Para **n** eventos necesito **n** Promises o cambiar el modelo. Un canal es naturalmente continuo.

P5: ¿Cómo separarías lógica pura de I/O en una función que busca usuarios en DB y calcula un descuento? > Respuesta esperada: La llamada a DB es I/O (efecto). El cálculo del descuento sobre el resultado es lógica pura. Separación: `fetchUser(id): Promise<User>` (I/O) → `calcularDescuento(user: User, pct: number): number` (pura). La función que los orquesta puede ser `async` pero llama a la pura como subfunción.

1.8 Riesgos frecuentes y mitigación

Riesgo 1: El grupo se detiene en la sintaxis de Clojure (paréntesis, macros) - Señal: preguntas del tipo “¿por qué hay dos paréntesis ahí?” que detienen el flujo. - Mitigación: decir explícitamente “la sintaxis no es el foco hoy — el concepto de transformación es el mismo que en TS”. Mostrar el equivalente TS al lado. Si persiste, omitir el código Clojure y trabajar solo con la descripción conceptual en esa sección.

Riesgo 2: Confusión entre asincronía (`Promise`) y concurrencia segura (STM/canal) - Señal: afirmaciones como “con `async/await` no hay race conditions”. - Mitigación: aclarar que en JS no hay threads de usuario, por eso no hay race conditions en ese sentido. Pero en Node.js servidor, múltiples requests comparten el mismo proceso y pueden intercalarse en puntos de `await`. Mostrar el ejemplo del contador compartido con dos `async` funciones que lo incrementan.

Riesgo 3: Sobreingeniería — querer usar `Result`, transducers y `core.async` para todo - Señal: diseños de taller con 4+ tipos de wrappers para un pipeline de 3 pasos. - Mitigación: preguntar explícitamente “¿el costo de leer este código es menor que el beneficio?” Recordar la regla: si una **función pura** y un `try-catch` exterior es suficiente, usar eso. Las abstracciones tienen costo de onboarding.

Riesgo 4: El taller se traba en TypeScript y no se llega a la comparativa - Señal: a los 15 minutos del taller un grupo no tiene pipeline funcional. - Mitigación: dar la solución de TypeScript en papel para que ese grupo avance. El objetivo del taller no es encontrar la solución — es **comparar** las dos soluciones. No sacrificar F-38 por el tiempo de implementación.

Riesgo 5: Alumnos confunden `map` sobre `Result` (monáda) con `map` sobre array - Señal:

código como `resultado.map(x => x * 2)` siendo que `resultado` es un `Result`, no un array. - Mitigación: clarificar que `Result` no tiene `.map()` nativo en TypeScript base — necesitamos la función `mapResult()` que definimos o una librería. Esta es la motivación para el tema siguiente (mónadas).

1.9 Material de respaldo

- Guía del alumno para profundización: `guia-estudio.md`
- Minuta para cronometraje y transición: `minuta.md`
- Filminas para conducción visual: `filminas.md`

Nota de fuentes: No se detectaron PDFs de referencia específicos del tema 04 en la carpeta material al momento de esta generación.